Source Code Metrics for Software Defects Prediction

Dominik Arne Rebro
Masaryk University
Brno, Czech Republic
domco28@mail.muni.cz

Bruno Rossi
Masaryk University
Brno, Czech Republic
brossi@mail.muni.cz

Stanislav Chren
Masaryk University
Brno, Czech Republic
chren@mail.muni.cz

ABSTRACT

In current research, there are contrasting results about the applicability of software source code metrics as features for defect prediction models. The goal of the paper is to evaluate the adoption of software metrics in models for software defect prediction, identifying the impact of individual source code metrics. With an empirical study on 275 release versions of 39 Java projects mined from GitHub, we compute 12 software metrics and collect software defect information. We train and compare three defect classification models. The results across all projects indicate that Decision Tree (DT) and Random Forest (RF) classifiers show the best results. Among the highest-performing individual metrics are NOC, NPA, DIT, and LCOM5. While other metrics, such as CBO, do not bring significant improvements to the models.

CCS CONCEPTS

• **Software and its engineering** → **Maintaining software**; **Software defect analysis**;

KEYWORDS

Software Defect Prediction, Software Metrics, Mining Software Repositories, Software Quality

ACM Reference Format:
Dominik Arne Rebro, Bruno Rossi, and Stanislav Chren. 2023. Source Code Metrics for Software Defects Prediction. In *The 38th ACM/SIGAPP Symposium on Applied Computing (SAC '23), March 27-March 31, 2023, Tallinn, Estonia.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3555776.3577809

1 INTRODUCTION

One of the crucial focuses of Software Engineering is the detection and correction of defects emerging in software systems. One approach that has been widely adopted in research is to try to predict software defects by analyzing the history of projects to identify parts of the source code that can be considered more prone to defects in the future – utilizing the so-called software defects prediction models [13].

However, while the results from existing defect prediction studies [4–7, 10, 12, 13] showcase that software metrics can be helpful in software defect prediction models, there is not an agreed view about which metrics are the best (and worse) to be used in a defect prediction model. Furthermore, the majority of the tools used in the related works are mostly deprecated or publicly unavailable. Therefore, the respective bug prediction databases created, even if public, can not be recomputed or extended with new projects (apart from Palomba et al. [13] that provides a replication package) – making the whole replication complicated if not unfeasible.

SAC '23, March 27-March 31, 2023, Tallinn, Estonia

ACM ISBN 978-1-4503-9517-5/23/03.
https://doi.org/10.1145/3555776.3577809

2 BACKGROUND & RELATED WORK

Software defect prediction model refers to statistical models that attempt to predict potential software defects based on the test data provided [11]. Given the mathematical nature of software metrics, they are suitable for input for defect prediction models. Prediction models consist of independent variables (software metrics) and a dependent variable indicating defectiveness. Multiple machine learning techniques are viable for defect prediction, including regression, classification, and clustering. Classification uses input data referred to as a training set, which consists of objects with known class labels. The goal of a classification algorithm is to analyze the training set to develop a model that can be used to classify objects with unknown labels. In this paper, we apply the three most popular classification algorithms used for defect prediction according to [21]: *Naive Bayes* (NB) [17], *Decision Tree* (DT) [14] and *Random Forest* (RF) [8]. We survey the existing body of work in the area of software metric-based defect prediction in Table 1.

3 EXPERIMENTAL DESIGN

We have two main Research Questions (RQs) in this research.

RQ1. *What is the impact of code metrics on software defect prediction?* The rationale of RQ1 is about evaluating how the inclusion of software metrics (and their specific groupings) can improve the software defect prediction results.

RQ2. *Which metrics are the most suitable predictors?* The rationale of RQ2 is about identifying which source-code level software metrics considered in this article are the best (and worse) predictors in the context of software defect prediction.

Data Source. To build the dataset for this study, we opted for mining the GitHub repositories. With open source projects, different metrics and prediction models can be tested on the same source code, allowing for better comparison of the results.

Project selection. In total 39 suitable Java projects, consisting of 275 versions, were selected for our dataset. The details of project selection process, together with all the projects are available on Figshare [16] – including statistics regarding the repository size, development team size, and bug-tracking information. More details are available also in [15].

Metrics selection. For the defect prediction analysis, the following 12 metrics were selected: (1) baseline metric LOC indicates the source code size for each class; (2) The CK [3] metric suite consists of six object-oriented metrics WMC, DIT, NOC, RFC, LCOM5

Table 1: Overview of metrics and prediction models in related work (full list of acronyms available in [16])

Ref	Year	Metrics	Prediction model
[6]	2020	**Source code metrics:** 60 class-level complexity, size and object-oriented metrics	Deep neural networks
[13]	2019	**Source code metrics:** ATFD, ATLD, CC, CDISP, CINT, CM, FanOut, LOC, LOCNAMM, MaMCL, MAXNESTING, MeMCL, NMCS, NOAM, NOLV, NOMNAMM, NPA, TCC, WOC **Code smell metric:** Code smell intensity	Simple Logistic, Decision Tree Majority, Naive Bayes, Logistic Regression
[18]	2013	**Source code metrics:** LOC, MLOC, PAR, NOF, NOM, NOC, CC, DIT, LCOM, NOT, WMC **Process metrics:** PRE, Churn **Antipattern metrics:** ANA, ACM, ARL, and ACPD	Various intra-system and cross-system
[1]	2011	**Explored metrics:** log(KLOC), Prior faults, Prior changes, Prior changed, Prior developers, Prior lines added/deleted/-modified	Binomial regression
[10]	2010	**Source code metrics:** WMC, DIT, NOC, RFC, LCOM, CBO, LCOM3, NPM, DAM, MOA, MFA, CAM, IC, CBM, AMC, Ca, Ce, CC, LOC	Kohonen's neural network
[4]	2010	**Source code metrics:** WMC, DIT, NOC, RFC, LCOM, CBO,LOC, FanIn, FanOut, NOA, NPA, NOPRA, NOAI, NOM, NPM, NOPRM, NOMI **Change metrics:** EDHCM, LDHCM, LGDHCM	Generalized linear regression
[9]	2009	**Change complexity metrics:** BCC, ECC, HCM	Statistical linear regression
[5]	2001	**Inheritance metrics:** DIT, NOC **Coupling metrics:** ACAIC, ACMIC, DCAEC, DCMEC, OCAIC, OCAEC, OCMIC, OCMEC	Calibrated logistic regression

(a modified version of the original LCOM metric), CBO; (3) The OTHER metric suite is consisting of five additional object-oriented metrics NPA, NPM, NLE, CBOI, CD.

Data collection. For implementing the whole data collection, processing and defect prediction process, we used Python libraries PyGithub, PyDriller and scikit-learn. An overview of the whole process is presented in Fig. 1. We employed an approach for linking fault information to system versions by [1, 9]. This approach allows for accumulating numerous bugs into a single version, the source code of which can be analyzed to calculate software metrics. Due to the object-oriented nature of the projects considered and class-level metrics being found more effective than file-level metrics, the dataset will be created considering only class-level metrics by using the static code analysis tool SourceMeter. By parsing diff files available for all files modified in a commit, we can learn which specific lines of a file were modified. With discovered source code positions of all classes, we can match the modified lines to specific classes and mark them as defective.

Data processing. Defect prediction models can suffer from multicollinearity, where two or more metrics of the model are highly correlated. Multicollinearity distorts feature importance analysis related to **RQ2**. In our case, feature importance values for each metric were computed using a 10-fold cross-validated permutation feature importance [2] model, defined as the decrease in the model score when a single feature value is randomly shuffled [2].

To address this issue, the Variation Inflation Factor (VIF) [19] is usually used in related research [13, 18]. By calculating the VIF value for each metric, it is possible to detect variables suffering from multicollinearity. As suggested in [18] the threshold of VIF was set to 2.5 for further investigation, while VIF values above 10 would indicate serious multicollinearity. In our study, the VIF values of all metrics were computed for each project. In a couple of outlying cases, some metrics achieved VIF close to 10, indicating significant multicollinearity. We concluded that there will probably always be an outlier project for any metric combination that suffers from high multicollinearity. In such cases, we simply disregarded these outlier projects from **RQ2** feature importance analysis.

Defect Prediction. We treat the prediction problem as a binary classification problem, assigning each class into the $defective$ or $not\ defective$ category. The processed dataset is divided into a training and testing subsets. The training set is used to fit (train) the prediction model, meaning the algorithm is learning how to predict the dependent variable. The testing set predicts dependent variable values and validates them based on scoring metrics such as precision, recall, F-measure, or AUC-ROC. For a better estimate of our models' performance, 10-fold cross-validation was employed by randomly splitting the data into 10 stratified folds, with 90% of the data used for training and 10% for testing.

4 RESULTS

4.1 Impact of code metrics (RQ1)

Regarding the performance of our defect prediction models, we evaluated the results separately for each metric suite to compare their effectiveness. For each scoring metric, we provide boxplots describing the distribution of results of each model. F-measure is computed taking into account the defective (minority) class as we believe this is the most useful scenario for defects prediction. AUC-ROC is instead computed using weighted averaging per class.

Looking at the F-measure distribution in Fig. 2, we can observe that in every metrics suite models, DT and RF achieve very similar results, while NB is performing significantly worse overall. This confirms previous research suggesting the suitability of DT and RF models for defect prediction [6, 20]. By comparing the interquartile range (dispersion) and median (location) of boxplots of models DT and RF, we can infer that using our defect dataset, the DT is slightly superior across all metric suites to RF in terms of F-measure – however, such differences are not statistically significant. Running Mann-Whitney U Test comparing the mean ranks, the differences are statistically significant for all the cases of NB vs DT and RF – while differences between DT and RF are not statistically significant. Running Kruskal-Wallis One-Way ANOVA on all the groups indicates statistically significant differences for all the four cases (LOC, CK, OTHER, CK+OTHER).

Comparing the F-measure between metric suites, as expected, LOC achieved the worst and least dispersed results. The differences between the distribution of results for suites CK, OTHER, and CK+OTHER are much more subtle. Overall, we found out similar results. While in [13] achieved F-measures are mostly between 50% and 80%, in our case the results are much lower due to the fact that F-measure was computed on the defective (minority) class – this explains F-measure lower than 50% in some cases.

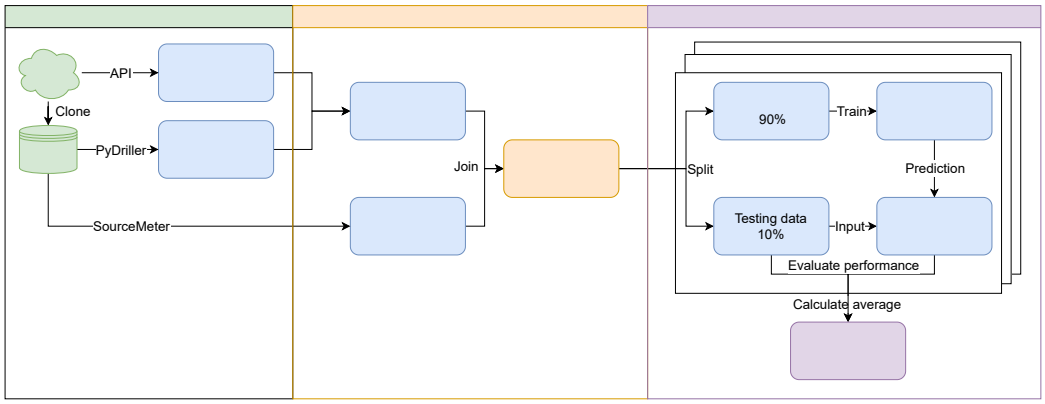

Figure 1: Overview of the experimental process

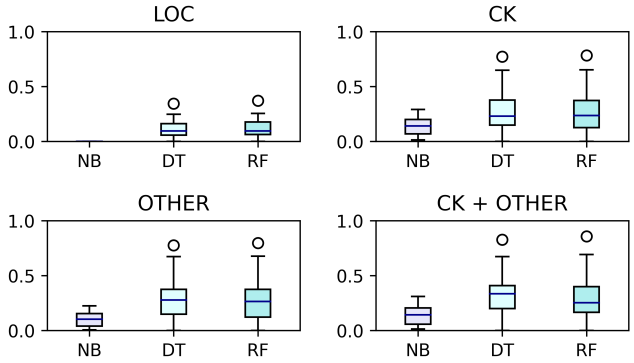

Figure 2: F-measure distribution by metric suites

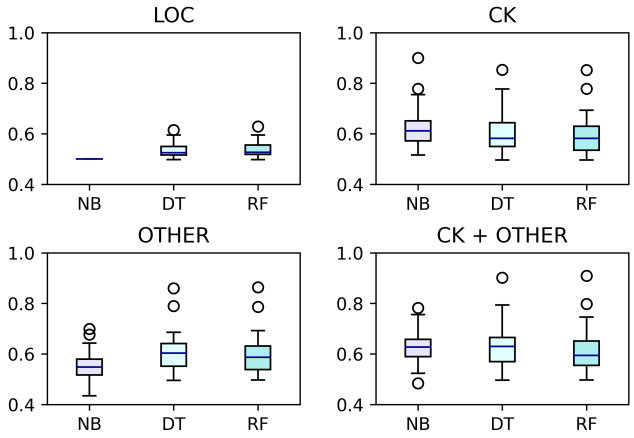

Figure 3: AUC-ROC distribution by metric suites

The AUC-ROC scores found in Fig. 3, reports slightly different results than the F-measure. In the case of LOC and OTHER suites the NB model is still underperforming. However, in suites CK and CK+OTHER, the NB model demonstrates slightly superior performance. However, it is essential to note that the results for NB models are marginally more unstable, also achieving AUC-ROC marginally under 50%, which brings concerns about the suitability of NB for defect prediction. Running Mann-Whitney U Test comparing the mean ranks, the differences are statistically significant for LOC, CK, OTHER in the cases of NB vs DT and RF but not significant in CK+OTHER case. Differences between DT and RF are not statistically significant in all cases. Running Kruskal-Wallis One-Way ANOVA on all the groups indicates statistically significant differences on two cases out of four (LOC, OTHER).

In terms of comparing AUC-ROC between metric suites, we can conclude that LOC is performing the worst, while other metrics suites CK, OTHER, CK+OTHER are similarly effective for defects prediction.

Overall, the achieved AUC-ROC scores are mostly between 50% and 75%, which is marginally worse than the results in some of the related research [6, 13], which achieved AUC-ROC scores ranging from around 60% to 80%.

RQ1 Summary

For defects prediction, the best results were achieved by DT and RF models. Using CK, OTHER, and CK+OTHER metric suites provides similar results, but improvements over standard LOCs metric.

4.2 Most suitable defect predictors (RQ2)

To identify which metrics had the most positive influence on the defect prediction, we inspect each metric's distributions of feature importance ranks for all suitable projects. The boxplots of metric ranks for every project in the aggregated form are shown in Fig. 4. The individual boxplots for each project are available on Figshare [16].

While carefully analyzing the distributions of feature importance ranks for each metric across all projects, we observed multiple trends. NOC is by far the best-performing metric, being ranked first across the vast majority of instances. Although less stable, metrics NPA, DIT, and LCOM5 tend to rank between 2nd and 4th. These

metrics, together with NOC, consistently have a significant positive influence on the defect prediction models.

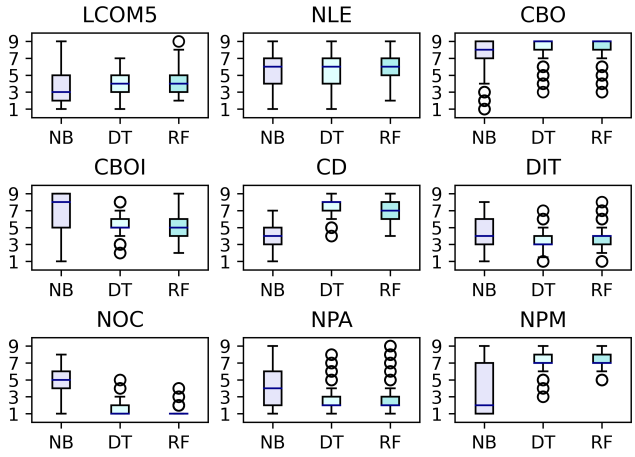

Figure 4: Rankings for all metrics (the lower the better)

Another stable trend observed is that CBO performed consistently worst, ranked last across most projects. With slightly more variability metrics, CD and NPM consistently achieved ranks 6^{th} to 8^{th}. This indicated that these metrics lack predictive power and contribute less to the overall defect prediction results. On the other hand, it could still be because of prevalent multicollinearity, which we addressed when processing the data.

RQ2 Summary

The best-performing metric across all projects is NOC, followed by NPA, DIT, and LCOM5. The worst-performing metric across all projects was CBO, followed by CD and NPM.

5 CONCLUSION

In this paper, we looked at the impact of software source code metrics on defect prediction models. We evaluated the performance of three classifiers in terms of several accuracy metrics and then the rankings of the metrics in terms of contribution to the prediction results. We mined software projects from GitHub data, building a dataset of 39 Java projects (275 release versions in 6-months ranges). The creation process integrated software defects information and project source code used to calculate 12 class-level software metrics and identify the classes containing software defects. Overall, we can report that DT and RF had the best results compared to NB and that using software metrics for defect prediction improves the performance of the classifiers. Related to the metrics with the most positive influence on defects prediction, we found that metrics NOC, NPA, DIT, and LCOM5 were consistently ranked as the best across all projects. Conversely, the metrics CBO, CD, and NPM were classified as the worst predictors.

ACKNOWLEDGMENTS

The work was supported by ERDF "CyberSecurity, CyberCrime and Critical Information Infrastructures Center of Excellence" (No. CZ.02.1.01/0.0/0.0/16_019/0000822).

REFERENCES

[1] Robert M. Bell, Thomas J. Ostrand, and Elaine J. Weyuker. 2011. Does Measuring Code Change Improve Fault Prediction?. In *Proceedings of the 7th International Conference on Predictive Models in Software Engineering (Promise '11)*. Association for Computing Machinery, New York, NY, USA, Article 2, 8 pages.

[2] Leo Breiman. 2001. Random forests. *Machine learning* 45, 1 (2001), 5–32.

[3] S.R. Chidamber and C.F. Kemerer. 1994. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering* 20, 6 (June 1994), 476–493. https://doi.org/10.1109/32.295895

[4] Marco D'Ambros, Michele Lanza, and Romain Robbes. 2010. An extensive comparison of bug prediction approaches. In *2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010)*. IEEE, 31–41. https://doi.org/10.1109/MSR.2010.5463279

[5] Khaled El Emam, Walcelio Melo, and Javam C. Machado. 2001. The Prediction of Faulty Classes Using Object-Oriented Design Metrics. *J. Syst. Softw.* 56, 1 (feb 2001), 63–75. https://doi.org/10.1016/S0164-1212(00)00086-8

[6] Rudolf Ferenc, Dénes Bán, Tamás Grósz, and Tibor Gyimóthy. 2020. Deep learning in static, metric-based bug prediction. *Array* 6 (2020), 100021. https://doi.org/10.1016/j.array.2020.100021

[7] T.L. Graves, A.F. Karr, J.S. Marron, and H. Siy. 2000. Predicting fault incidence using software change history. *IEEE Transactions on Software Engineering* 26, 7 (July 2000), 653–661. https://doi.org/10.1109/32.859533

[8] Péter Gyimesi. 2017. Automatic Calculation of Process Metrics and their Bug Prediction Capabilities. *Acta Cybernetica* 23 (01 2017), 537–559.

[9] Ahmed E. Hassan. 2009. Predicting faults using the complexity of code changes. In *2009 IEEE 31st International Conference on Software Engineering*. IEEE, 78–88. https://doi.org/10.1109/ICSE.2009.5070510

[10] Marian Jureczko and Lech Madeyski. 2010. Towards Identifying Software Project Clusters with Regard to Defect Prediction. In *Proceedings of the 6th International Conference on Predictive Models in Software Engineering (PROMISE '10)*. Association for Computing Machinery, New York, NY, USA, Article 9, 10 pages.

[11] Zhiqiang Li, Xiao-Yuan Jing, and Xiaoke Zhu. 2018. Progress on approaches to software defect prediction. *Iet Software* 12, 3 (2018), 161–175.

[12] Ran Mo, Shaozhi Wei, Qiong Feng, and Zengyang Li. 2022. An exploratory study of bug prediction at the method level. *Information and Software Technology* 144 (2022), 106794. https://doi.org/10.1016/j.infsof.2021.106794

[13] Fabio Palomba, Marco Zanoni, Francesca Arcelli Fontana, Andrea De Lucia, and Rocco Oliveto. 2019. Toward a Smell-Aware Bug Prediction Model. *IEEE Transactions on Software Engineering* 45, 2 (2019), 194–218. https://doi.org/10.1109/TSE.2017.2770122

[14] Manjula.C.M. Prasad, Lilly Florence, and Arti Arya. 2015. A Study on Software Metrics based Software Defect Prediction using Data Mining and Machine Learning Techniques. *International Journal of Database Theory and Application* 7 (06 2015), 179–190. https://doi.org/10.14257/ijdta.2015.8.3.15

[15] Dominik Arne Rebro. 2022 [cit. 2022-12-28]. *Software Metrics for Software Defects Prediction [online]*. Master's thesis. Masaryk University, Faculty of Informatics, Brno. Supervisor : Bruno Rossi.

[16] Dominik Arne Rebro, Bruno Rossi, and Stanislav Chren. 2022. Software Metrics for Defects Prediction Dataset. (2022). https://doi.org/10.6084/m9.figshare.21150994.v1

[17] Iqbal H. Sarker. 2021. Machine Learning: Algorithms, Real-World Applications and Research Directions. *SN Computer Science* 2, 3 (22 Mar 2021), 160. https://doi.org/10.1007/s42979-021-00592-x

[18] Seyyed Ehsan Salamati Taba, Foutse Khomh, Ying Zou, Ahmed E. Hassan, and Meiyappan Nagappan. 2013. Predicting Bugs Using Antipatterns. In *2013 IEEE International Conference on Software Maintenance*. IEEE, 270–279. https://doi.org/10.1109/ICSM.2013.38

[19] Christopher Glen Thompson, Rae Seon Kim, Ariel M Aloe, and Betsy Jane Becker. 2017. Extracting the variance inflation factor and other multicollinearity diagnostics from typical regression results. *Basic and Applied Social Psychology* 39, 2 (2017), 81–90.

[20] Zoltán Tóth, Péter Gyimesi, and Rudolf Ferenc. 2016. A Public Bug Database of GitHub Projects and Its Application in Bug Prediction. In *Proceedings of the 16th International Conference on Computational Science and Its Applications (ICCSA 2016)*. Springer International Publishing, Beijing, China, 625–638.

[21] Romi Satria Wahono. 2015. A systematic literature review of software defect prediction. *Journal of software engineering* 1, 1 (2015), 1–16.